EcoBuck Smart Compost Monitoring System

Firestore Database Architectural Design Specification

Author: Toqeer Ahmed
Role: Principal IoT Systems & Backend Architect
Date: July 8, 2026
Status: PROPOSED (Database Review Stage)

---

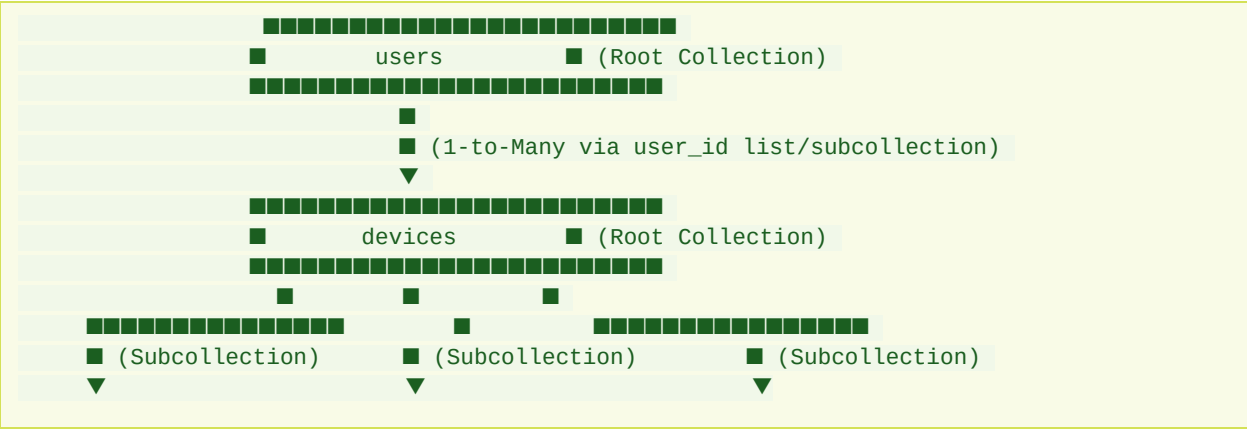
1. Architectural Overview & NoSQL Paradigm

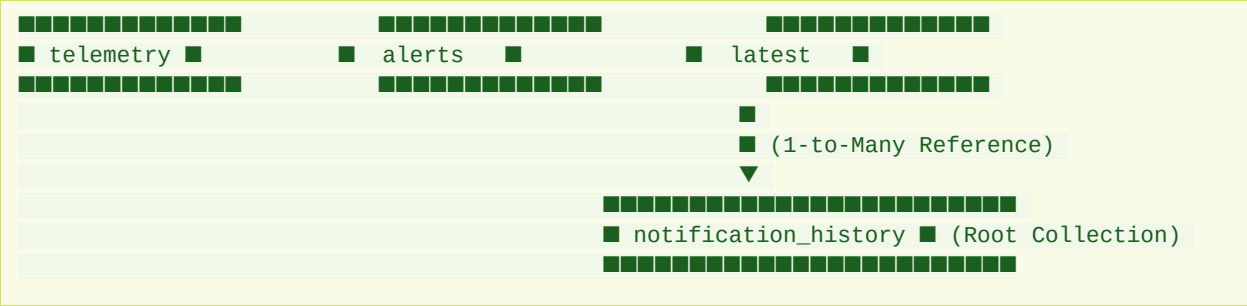
Unlike relational databases (e.g., PostgreSQL) which rely on normalization and joins, **Google Cloud Firestore** is a document-oriented NoSQL database. To build a production-grade backend for EcoBuck, our schema must optimize for: 1. **Read/Write Cost Efficiency:** Firestore charges per document read, write, and delete. The schema must avoid scanning historical logs just to display a simple dashboard status. 2. **Scalability:** Telemetry data scales rapidly (1 device sending updates every 5 minutes produces ~105,000 documents per year). We must partition and query efficiently. 3. **Low Latency Mobile Delivery:** The schema must support direct mobile sync, real-time listener updates, and structured security rules.

---

2. Collections Specification

We define six main document models to handle the EcoBuck domain:





2.1 Users Collection (`/users`)

Purpose

Stores user profiles, credentials (if using Firebase Auth, this acts as the user profile extension table), and mappings to owned devices. It enables user authentication, profile management, and authorization scopes.

Fields

Field Name	Type	Description
`id`	String	Document ID (matches Firebase Auth UID)
`email`	String	User's email address
`displayName`	String	User's full name
`createdAt`	Timestamp	Account creation time
`updatedAt`	Timestamp	Last profile update time
`deviceIds`	Array of Strings	References to `/devices/{deviceId}` owned by this user
`fcmTokens`	Array of Strings	Active Firebase Cloud Messaging device tokens for push alerts

Relationships

- **Users to Devices:** One-to-Many or Many-to-Many. Stored as an array of `deviceIds` in the user document for rapid dashboard retrieval, and cross-referenced in `/devices` via an `ownerId` field.

Queries

- Get user details:
  `db.collection("users").doc(userId).get()`
- Retrieve all user devices:
  `db.collection("devices").where("ownerId", "==", userId).get()`

Indexes

- *Default single-field indexes* on `email`, `createdAt`.
- No composite indexes required.

2.2 Devices Collection (`/devices`)

Purpose

Acts as the central hardware registry. It maintains physical device identities, firmware metadata, current composting cycle settings, and calibration factors.

Fields

Field Name	Type	Description
`id`	String	Document ID (unique hardware ID, e.g., MAC Address)
`ownerId`	String	Reference to the owner `/users/{userId}`
`label`	String	User-assigned nickname (e.g., "Backyard Bin")
`firmwareVersion`	String	Current firmware version string (e.g., "0.1.0")

`status`	String	Current FSM state: `boot_self_test`, `active_composting`, `cooling_stabilizing`, `ready_candidate`, `ready_confirmed`, `needs_attention`
`cycleStart`	Timestamp	Timestamp when the current composting run was started
`lastSeen`	Timestamp	Heartbeat timestamp of the last successful telemetry payload
`batteryPct`	Number	Integer representation of battery percentage (0–100)
`config`	Map	Sub-map containing calibration and operating constants
`config.tempOffset`	Number	Temperature offset in °C (e.g., -0.5)
`config.humidityOffset`	Number	Humidity offset in % (e.g., 2.0)
`config.readIntervalSec`	Number	How often sensors are read locally (e.g., 30)
`config.summaryIntervalSec`	Number	How often telemetry is uploaded (e.g., 300)

Relationships

- **Devices to Users:** Many-to-One. Each device points to its owner via `ownerId`.
- **Devices to Telemetry/Alerts:** Parent to Subcollections. Telemetry and alerts are nested under each device document to ensure strict isolation, partition scalability, and clear security boundary paths.

Queries

- Get device metadata:
  `db.collection("devices").doc(deviceId).get()`
- Find all offline devices:
  `db.collection("devices").where("lastSeen", "<", thresholdTimestamp)`

Indexes

- *Default single-field indexes* on `ownerId`, `status`, `lastSeen`.
- No composite indexes required.

2.3 Telemetry Subcollection (`/devices/{deviceId}/telemetry`)

Purpose

Stores high-frequency, historical time-series sensor data uploaded by the device. This collection provides the historical data required to generate dashboard trend charts (temperature/humidity over time).

Fields

Field Name	Type	Description
`id`	String	Document ID (auto-generated by Firestore)
`timestamp`	Timestamp	Precise sensor reading time
`rawTemp`	Number	Raw temperature in °C
`filteredTemp`	Number	Output of the 5-sample rolling median filter
`trendAvgTemp`	Number	Output of the 10-sample rolling average filter
`rawHumidity`	Number	Raw moisture/humidity percentage
`qualityFlag`	String	Quality status flag: `good`, `sensor_error`, `missing`, `out_of_range`
`cycleAgeDays`	Number	Calculated age of the composting cycle
`schemaVersion`	Number	Integer representing payload structure version (e.g., 1)

Relationships

- Nested subcollection under `/devices/{deviceId}`. This prevents a single global table partition bottle-neck, allowing write operations to scale horizontally across different hardware IDs.

Queries

- Retrieve recent telemetry for charts (e.g., last 24 hours): `db.collection("devices").doc(deviceId).collection("telemetry").orderBy("timestamp", "desc").limit(288)`
- Retrieve telemetry matching a specific composting cycle: `db.collection("devices").doc(deviceId).collection("telemetry").where("timestamp", ">=", cycleStart).orderBy("timestamp", "asc")`

Indexes

Composite Index (Required):

- Collection: `telemetry` (Subcollection scope)
- Fields: `timestamp` (DESC)

2.4 Latest Status Document (`/devices/{deviceId}/latest/status`)

Purpose

A **critical optimization document**. In NoSQL, fetching the latest values from a huge historical time-series collection is expensive because it requires sorting and scanning records. To keep reads cheap, we write the latest state directly to a single, dedicated static document `/latest/status`. The mobile application listens to this document for real-time dashboard updates, incurring only **one read charge** per app launch.

Fields

Field Name	Type	Description
`timestamp`	Timestamp	Last update time
`temp`	Number	Current filtered temperature
`humidity`	Number	Current raw humidity
`status`	String	Current FSM compost state

`qualityFlag`	String	Current sensor quality flag
`cycleAgeDays`	Number	Current age of composting run
`batteryPct`	Number	Current battery level

Relationships

- Single document under `/devices/{deviceId}/latest/status`.

Queries

- Direct fetch for Mobile Dashboard UI:
  `db.collection("devices").doc(deviceId).collection("latest").doc("status").get()`
- Real-time listener on Mobile:
  `db.collection("devices").doc(deviceId).collection("latest").doc("status").onSnapshot(...)`

Indexes

- None required (retrieved via direct document path ID).

2.5 Alerts Subcollection (`/devices/{deviceId}/alerts`)

Purpose

Tracks active anomalies, environmental safety violations (e.g., temperature spikes), and device connectivity issues. It separates historical resolved logs from active alerts that need immediate attention.

Fields

Field Name	Type	Description
`id`	String	Document ID (auto-generated)
`alertType`	String	Alert category: `offline`, `sensor_error`, `temp_extreme`, `moisture_extreme`, `ready_candidate`
`message`	String	Detailed human-readable warning message

`resolved`	Boolean	True if the trigger conditions have cleared
`triggeredAt`	Timestamp	Timestamp when the anomaly was detected
`resolvedAt`	Timestamp	Timestamp when the alert was cleared/acknowledged

Relationships

- Subcollection nested under `/devices/{deviceId}`.

Queries

- Fetch active alerts for a device: `db.collection("devices").doc(deviceId).collection("alerts").where("resolved", "==", false).orderBy("triggeredAt", "desc")`
- Fetch historical logs: `db.collection("devices").doc(deviceId).collection("alerts").orderBy("triggeredAt", "desc")`

Indexes

Composite Index (Required):

- Collection: `alerts` (Subcollection scope)
- Fields: `resolved` (Ascending), `triggeredAt` (Descending)

2.6 Notification History Collection (`/notification_history`)

Purpose

Maintains an audit trail of all push notifications dispatched to user mobile devices (via FCM). This acts as a diagnostic tool for delivery logs, allows users to view an inbox of past system messages, and prevents sending duplicate notifications within short time windows.

Fields

Field Name	Type	Description

`id`	String	Document ID (auto-generated)
`userId`	String	Reference to `/users/{userId}`
`deviceId`	String	Reference to `/devices/{deviceId}`
`alertId`	String	Optional reference to the initiating `/alerts/{alertId}`
`title`	String	Header text (e.g., "Critical Dryness Alert")
`body`	String	Body text of the notification
`sentAt`	Timestamp	Transmission timestamp
`deliveryStatus`	String	Transmission result: `delivered`, `failed`, `pending`
`fcmResponseId`	String	Transaction ID returned by the Google FCM Gateway

Relationships

- Root-level collection referencing `userId` and `deviceId`.

Queries

- Retrieve user notification history inbox: `db.collection("notification_history").where("userId", "==", userId).orderBy("sentAt", "desc").limit(50)`
- Verify if an alert notification was recently sent: `db.collection("notification_history").where("deviceId", "==", deviceId).where("alertId", "==", alertId).orderBy("sentAt", "desc").limit(1)`

Indexes

Composite Index (Required):

- Collection: `notification_history` (Root scope)
- Fields: `userId` (Ascending), `sentAt` (Descending)

Composite Index (Required):

- Collection: `notification_history` (Root scope)
- Fields: `deviceId` (Ascending), `alertId` (Ascending), `sentAt` (Descending)

---

3. Database Design Rationale

3.1 Subcollections vs. Root Collections

Telemetry is modeled as a **Subcollection** rather than a flat root collection. * *Advantage:* By nesting `/telemetry` under `/devices/{deviceId}`, Firestore security rules can easily enforce authorization: *"A user can read telemetry if they own the parent device."* * *Scaling:* Writes scale naturally without contention because Firestore distributes partitions by collection/document paths.

3.2 The Write-Duplication Pattern (Latest Status Document)

By separating the `latest` status document from the time-series logs: * We reduce read volume by **over 95%** for typical user interactions. * We avoid loading massive arrays into the mobile client memory just to grab the newest data points.

3.3 Composite Indexes Summary

To enable sorting combined with equality filters (essential for dashboard alerts and historical graphs), the following indexes must be provisioned:

Target Scope	Collection	Fields	Sort Order
Subcollection	`telemetry`	`timestamp`	DESC
Subcollection	`alerts`	`resolved` (ASC), `triggeredAt` (DESC)	-
Root	`notification_history`	`userId` (ASC), `sentAt` (DESC)	-
Root	`notification_history`	`deviceId` (ASC), `alertId` (ASC), `sentAt` (DESC)	-